

CS-2009 Design and Analysis of Algorithms-Fall 2025

Assignment#2

Due Date and Time: 23rd October, 2025 (11:30 AM)

Total: 113 marks

Instructions:

1. Late assignment will not be accepted
2. Only handwritten attempt will be graded, i.e., printed attempts will not be graded.
3. There will be no credit if the given requirements are changed
4. Please mention the question number and its part (if any) before writing down its solution. Use the same conventions used in the assignment's document.
5. Don't plagiarize. Plagiarism would lead to straight zero in the assignment regardless of the percentage copied.
6. Also submit scanned copies of your assignment on GCR. 20% penalty for not submitting it on GCR

Quick Sort

Question 1: Quick Sort is a divide-and-conquer sorting algorithm that relies on a *partition function* to divide the array into subarrays. The following question requires you to modify the standard partition function and write complete pseudo-code for Quick Sort in two different ways. [15]

- a.** The partition of an array must satisfy the following two conditions: Elements smaller than the pivot element must appear at index less than or equal to partition index. Elements larger than or equal to pivot element must appear at index greater than partition index. Partition index is equal to count of elements, strictly smaller than pivot, minus one. *Note: There might be more than one possible partition arrays.*

Input: arr[] = [5, 3, 8, 4, 2, 7, 1, 10]

Output: [1, 3, 2, 4, 8, 7, 5, 10]

Explanation: The partition index is 3 and pivot element is 5, all elements smaller than pivot element [1, 3, 2, 4] were arranged before partition index and elements larger than or equal to pivot [8, 7, 5, 10] were arranged after partition index. Name this function as PARTITION_A.

- b. Given an array `arr[]`, the task is to partition the array by assuming last element as pivot element. The partition of an array must satisfy the following two conditions: Elements smaller than the pivot element appear before pivot in the array. Elements larger than or equal to the pivot element appear after pivot in the array. Name this function as `PARTITION_B`.

Note: There might be more than one possible partition arrays.

Input: `arr[] = [5, 13, 6, 9, 12, 11, 8]`

Output: `[5, 6, 8, 13, 9, 12, 11]`

Explanation: All elements smaller than pivot element [5, 6] were arranged before it and elements larger than pivot [13, 9, 12, 11] were arranged after it.

Question 2: Demonstrate the operation Quick Sort with the `PARTITION_A` function on the array `A {13,19,9,5,12,8,7,4,11,2,6,21}`, showing the values of the array and auxiliary values after every iteration during the partition function. Also, show the array in the call stack. [5]

Question 3: The analysis of the expected running time of randomized quicksort assumes that all element values are distinct. In this problem, we examine what happens when they are not. Suppose that all element values are equal. What would be randomized quick sort's running time in this case? [5]

Question 4: The quicksort algorithm contains two recursive calls to itself. After quicksort calls partition, it recursively sorts the left subarray, and then it recursively sorts the right subarray. The second recursive call in quicksort is not really necessary; we can avoid it by using an iterative control structure. Modify the code without using the second recursive call and show its dry run on the following array to show that your algorithm correctly sorts the array. In the dry run, show your call stack and also the array at each call [10]

`A = [5, 3, 8, 4, 2, 7, 1, 10]`

Heap Sort

Question 5 : A d-ary heap is like a binary heap, but (with one possible exception) non-leaf nodes have d children instead of 2 children. [15]

- How would you represent a d-ary heap in an array?
- What is the height of a d-ary heap of n elements in terms of n and d?
- Give an efficient implementation of `EXTRACT-MAX` in a d-ary max-heap. Analyze its running time in terms of d and n. The code for `EXTRACT-MAX` of a binary max heap is provided.

```

HEAP-EXTRACT-MAX(A)
1  if A.heap-size < 1
2      error "heap underflow"
3  max = A[1]
4  A[1] = A[A.heap-size]
5  A.heap-size = A.heap-size - 1
6  MAX-HEAPIFY(A, 1)
7  return max

```

Question 6: We use array-based binary heaps along with the standard max_heapify procedure and Floyd's BuildHeap algorithm. The implementation is in-place, meaning it requires only $O(1)$ additional space beyond a constant number of temporary variables. All comparisons are limited to the standard relational operators $<$, $=$, and $>$, and tie-breaking is deterministic, for example, by always preferring the left child in case of a tie. **[10]**

a. Decide if it's possible (short proof or counterexample).

Does there exist an in-place BuildHeap that is stable on all inputs and runs in $O(n)$ time?

- If **yes**, give the algorithm and prove stability.
- If **no**, give an argument and a running example. *Hint:* Think about equal-key siblings during max_heapify and whether a fixed tie-break can preserve original order without remembering anything about the past.

b. Now relax in-place and design a stable BuildHeap.

Drop the in-place requirement and design a stable BuildHeap that runs in $O(n)$ time using $O(n)$ auxiliary space.

Provide pseudocode and a short correctness proof (why duplicates keep their original relative order in the heap and in the eventual heapsort output). For proof, dry-run your algorithm on the following array, $A=[5,1,3,3,2,5,3]$ naming equal elements as 3a, 3b, and so on.

Question 7: You are tasked with improving the traditional Heap Sort. **[15]**

Design a recursive sorting algorithm that:

- Uses two opposing heaps a max-heap for larger elements and a min-heap for smaller elements.
 - Recursively partitions the array into subheaps, using a TreeSwap procedure to ensure all smaller elements reside in the lower partition and larger elements in the upper partition.
 - Continues partitioning each subheap until all are too small to divide further.
-

Merge Sort/Insertion Sort

Question 8 : Design a stable, comparison-based sorting algorithm for an array of integers that achieves $\Theta(n \log n)$ worst-case time while remaining fast in practice. [20]

Requirements

1. Hybrid approach:
Use a simple quadratic-time method only on small segments to exploit locality; then iteratively combine adjacent sorted segments until the whole array is sorted.
2. Stability:
Equal elements must preserve their original relative order.
3. Iterative (non-recursive):
Write the algorithm without recursion.
4. Complexity & space:
 - o Worst-case time: $\Theta(n \log n)$.
 - o Extra space: up to $O(n)$ is allowed during the combining phase.
5. Tunable threshold:
Choose a small segment size K (a fixed constant you pick). Justify your choice briefly.

What to write

- A function to sort a small segment $[l..r]$ using a simple method suitable for tiny ranges.
- A function that stably combines two neighboring sorted segments into one larger sorted segment.
- A driver that:
 - o Partitions the array into small segments and sorts each;
 - o Then repeatedly combines neighboring segments in passes until the array is fully sorted.

Linear Sort

Question 9: You are given an array of strings, where different strings may have different numbers of characters, but the total number of characters over all the strings is n . Show how to sort the strings in $O(n)$ time. (Note that the desired order here is the standard alphabetical order; for example, $a < ab < b$.) [5]

Question 10: Prof. Madni decides to perform Bucket Sort on n items using n^2 buckets, while his student Ms. Sara Shams decides to perform Bucket Sort on n items using $n^{1/2}$ buckets. Assume that the items are uniformly distributed and that Ms. Sara's algorithm for sorting within a bucket takes $O(m^2)$ time when the bucket contains m items. [6]

a. Argue that Prof. Madni has made a poor choice in the number of buckets by analyzing the time complexity of Bucket Sort in his case.

b. Argue that Ms. Sara Shams has made a poor choice in the number of buckets by analyzing the time complexity of Bucket Sort in her case.

c. Argue that Ms. Sara Shams uses roughly the same amount of space as someone using n buckets.

Question 11: The counting sort variant we studied in class caters to positive integers. Change the algorithm of counting sort so it also caters to negative values in descending order. Perform a dry to the following array to show it works correctly. You have to show the final array of C after every loop you execute. [7]

[8, 5, -6, 7, 1, 4 , 5] Input

[8, 7, 6, 5, 5, 4, 1, -6] Output

Best of Luck 😊